

CAPSTONE

Capstone Projects

Nine ways to build, measure, and defend a small LLM system — pick one.

9 projects · each with a starter notebook, a starter prompt, and seed data



AT A GLANCE

The nine projects, grouped

Four families of task. Each project is measurable, and each comes with a notebook and seed data ready to run.



Classify & extract

P1 Support-ticket triage · P3 Structured extractor



Retrieve & ground

P2 Docs Q&A (RAG) · P4 Text-to-SQL with guardrails



Summarize & judge

P5 Summarizer · P9 Auto-grader · P8 Quality-vs-cost



Tools & safety

P6 Tool-using agent · P7 Injection red-team

PROJECTS

Capstone Projects · Prompt & Context Engineering

starter notebook + data included

Every starter follows the same shape

Open a notebook and you can run it immediately, then the TODOs turn it into a capstone.

1

Load the seed data

The first cell reads a small dataset from data/ — labelled tickets, a doc corpus, a toy database, and so on.

2

Run the starter prompt

A working prompt is already there. You get output on real examples before writing a line.

3

Score it

The notebook wires the data into the course eval harness, so you get a number, not a vibe.

4

Iterate (the TODOs)

Improve the prompt, add hard cases, add an injection test — and re-measure. Deliver via Git.

Support-ticket triage

TRACK A

- **Build**

Classify a ticket (billing / bug / other), extract product & severity, draft a reply.

- **Data**

tickets.jsonl — 24 tickets, each labelled with type, product, severity.

- **Evaluate**

Classification accuracy, then field-level extraction accuracy.

- **Injection**

A ticket that says 'ignore instructions' must not be obeyed.

starter prompt

```
System: you are a triage bot
Classify the ticket as
billing, bug, or other.
Reply with ONE lowercase word.
```

```
Ticket: "App crashes on export"
```

Grounded docs Q&A (RAG)

TRACK A

- **Build**

Answer over a corpus; cite sources; refuse when the answer isn't there.

- **Data**

handbook.md (16 passages) + rag_questions.jsonl (12, in & out of scope).

- **Evaluate**

Grounding, citation correctness, and correct refusal.

- **Injection**

A poisoned document that hides an instruction.

starter prompt

```
Answer using ONLY the context.  
Cite sources as [n].  
If the answer isn't there, say:  
"I do not know."
```

```
Context: {retrieved chunks}  
Question: {q}
```

Structured extractor

TRACK A

- **Build**

Turn messy text into validated JSON with a parse → validate → repair loop.

- **Data**

job_posts.jsonl — 18 posts labelled with title, seniority, remote.

- **Evaluate**

Field-level accuracy against the labels.

- **Skills**

Schema design, pydantic validation, repair-on-failure.

```
starter prompt + schema
```

```
Return ONLY JSON:
```

```
{ title: str,  
  seniority: "junior|mid|senior",  
  remote: bool }
```

```
Text: {messy job post}
```

```
# parse -> validate -> repair
```

Text-to-SQL with guardrails

TRACK A

- **Build**

NL questions become runnable SELECTs; destructive requests are refused.

- **Data**

shop.sql (customers + orders) + 8 questions with reference SQL + 5 unsafe ones.

- **Evaluate**

Execution accuracy (right rows) + a safety set that must be blocked.

- **Guardrail**

Reject any non-SELECT before it reaches the database.

starter prompt

```
# schema: customers, orders
Write ONE read-only SELECT
for the question.
Return ONLY the SQL.
Never INSERT/UPDATE/DELETE/DROP.
```

Question: {q}

Controllable summarizer

TRACK A

- **Build**

Summarize to an audience, length and format; extract action items.

- **Data**

transcripts.jsonl — 8 transcripts with reference summaries + actions.

- **Evaluate**

Checkable constraints (length) + an LLM-as-judge for faithfulness.

- **Watch**

Validate the judge against your own labels before trusting it.

starter prompt

```
Summarize for {audience}  
in <= 50 words, as 3 bullets.  
Then list action items.
```

Text: {transcript}

+ LLM-as-judge faithfulness

Tool-using agent

TRACK B

- **Build**

A ReAct-style loop that picks tools, reads results, and answers.

- **Data**

facts.json (12-fact KB) + 8 multi-step questions with expected tools.

- **Evaluate**

Tool-selection accuracy + end-task correctness.

- **Lesson**

The tool description IS the prompt — fix the words, not the wiring.

```
tools (the description is the prompt)
```

```
Tools:
```

```
  lookup(key)      -> fact
```

```
  calculator(expr) -> number
```

```
Think, choose a tool,  
observe, repeat, then answer.
```

```
Question: {multi-step q}
```

Injection red-team lab

TRACK B

- **Build**

Attack a bot, then layer guardrails and measure what still gets through.

- **Data**

attacks.jsonl — 12 attacks (direct / indirect / exfil / jailbreak + a control).

- **Evaluate**

Attack success rate, before vs after your defenses.

- **Honesty**

There is no perfect defense — report what still slips past.

```
attack battery
```

```
# 12 attacks:
```

```
direct / indirect / exfil /  
jailbreak (+ benign control)
```

```
success = "pwned" in reply  
measure rate before / after  
adding guardrails
```

Quality-vs-cost benchmark

MIXED

- **Build**

Compare prompt variants across a small local and a frontier model.

- **Data**

benchmark_task.jsonl — 15 topic-classification cases (finance/sports/tech).

- **Evaluate**

A quality / cost / latency table + a justified recommendation.

- **Fit**

More analysis than build — good for a measure-first team.

```
task + measurement
```

```
Classify as finance,  
sports, or tech.
```

```
# run on small vs frontier  
# log accuracy, tokens, latency  
# recommend a config
```

Rubric-based auto-grader

TRACK A

- **Build**

An LLM-as-judge for a concrete grading task, calibrated to humans.

- **Data**

graded_answers.jsonl — 15 answers with a human score (0-3).

- **Evaluate**

Agreement between the judge and the human scores.

- **Payoff**

Makes the 'validate your judge' lesson stick better than anything.

```
starter judge
```

```
Question: {q}
```

```
Rubric: score 0-3
```

```
Answer: "{a}"
```

```
Return ONLY JSON
```

```
{ score: int, reason: str }
```

```
# calibrate vs human labels
```

CHOOSING

How to choose

- **Safest first projects**

1 triage, 3 extractor, 5 summarizer — clear answers, no API keys.

- **Highest ceiling**

6 agent, 7 red-team — hosted model, more moving parts, strong teams only.

- **More analysis than build**

8 benchmark — for a team that would rather measure than engineer.

- **Whatever you pick**

Grow the seed data, add an injection test, deliver via pull requests and a tag.

PROJECTS

Capstone Projects · Prompt & Context Engineering

starter notebook + data included